ITCS 4102 / 5102 Final Project
Julia As a Fraud Detection Tool

Project Title	*Fraud Detection Using Julia*
Programming Language	*Language Learned: Julia* *Others Used: JavaScript, HTML*
Team Members	*Devin Washington, Lavashia Crump, Angel Vilar Nolasco*
Version Control Link	*https://github.com/DevinW2304/ITCS-4102*

1. Introduction and Motivation

1.1 Project Overview

This project is a small fraud detection demonstration built with a Julia-based backend and an HTML, CSS, and JavaScript frontend. The system reads a PaySim-style transaction dataset in CSV format and runs the transactions through a rule-based fraud detection model to identify suspicious behavior. A Node.js backend connects the frontend dashboard to the Julia script, allowing the browser to send a request, run the analysis, and display returned metrics, flagged transactions, and risk classifications. The goal of the project is to show how Julia can be used in a practical application that processes transaction records, evaluates fraud-related signals, and returns clear results through a working interface.

1.2 Motivation

We chose to create a fraud detection tool because it is a practical, growing problem area that fits well with a language like Julia. Digital transactions are now a regular part of everyday life, which creates more opportunities for suspicious or fraudulent activity. Because of the volume of modern transaction activity, fraud detection often depends on automated systems that can process large numbers of records and identify unusual patterns more efficiently than a manual review. This made fraud detection a strong topic for our semester project because it gave us a meaningful, real-world problem while also allowing us to build a data-focused system with structured records and measurable output. Our presentation noted that in 2025, the FTC reported about 3.0 million fraud reports and $15.9 billion in consumer losses, compared with more than $12 billion the previous year, underscoring the importance of the topic.

The topic also gave us a chance to create something more substantial than a basic demonstration, following merely the requirements. Instead of only showing syntax examples, we used Julia in a project that involved transaction parsing, account history tracking, weighted fraud scoring, and result classification. That made the project feel both relevant and technically meaningful, even though we were not able to fully maximize the computing abilities of Julia.

1.3 Why We Chose This Language

Initially, we chose Julia because we wanted to explore a language that could leverage advanced computational resources, including Centaurus at UNC Charlotte. Our original idea was to investigate whether we could build something at a scale that would integrate well with Julia's performance and parallel-processing strengths. As the semester progressed, we realized that building a project large enough to justify that approach was not realistic within one semester while also managing full course loads. Even so, we decided to continue with Julia because it still matched the kind of system we were building.

Our project needed to load transaction data, parse CSV records, apply repeated fraud checks, calculate scores, classify results, and return structured output to the frontend. Julia supported that well through readable syntax, typed structs, dictionaries, control structures, and functions that helped organize the program. In our implementation, Julia was used to model transactions, store customer and destination statistics, and apply the fraud-scoring rules that drive the project's classifications.

We also stayed with Julia because it allowed us to learn a language outside our usual stack while still producing a useful final project. It introduced us to a different programming style while remaining flexible enough

to integrate with the rest of the system through the Node.js backend and browser dashboard. That made Julia a worthwhile and appropriate choice for our semester project.

2. Language Specifications

2.1 Language Paradigm

Julia is a multi-paradigm language that supports imperative, functional, and object-oriented styles, with multiple dispatch serving as one of its most distinctive features. This flexibility allows programmers to use whatever style best fits the problem they are solving. In practice, an imperative approach in Julia often uses step-by-step instructions, loops, and mutable state, while a functional approach emphasizes functions, immutability, and cleaner separation of logic. For our project, this flexibility was useful because it allowed us to combine structured data handling with clear rule-based logic.

2.2 Historical Background

Julia was officially launched in February 2012 by Jeff Bezanson, Stefan Karpinski, Viral B. Shah, and Alan Edelman. It was designed to be a free, high-level language that also offered strong performance. The goal was to create a language that combined the speed usually associated with lower-level languages and the ease of use found in higher-level ones. Julia draws inspiration from several other languages, including MATLAB and R for mathematical and scientific expression, Lisp and Scheme for macros, and Python for readability and general ease of use.

2.3 Language Elements

Julia has a well defined set of elements including reserved words, primitive data types, and structured data types. Which include but are not limited to:

Category	Examples	Description
Reserved words	if, else, while, function, return, module	Keywords with predefined meanings are used to control the structure of the program.
Primitive data types	Int, Float64, Bool, Char	Basic built-in types representing simple values.
Structured data types	Array, Tuple, Dict, struct	Collections or user-defined types for organizing complex data.

2.4 Syntax Overview

JULIA syntax is designed to be readable and concise, while maintaining performance, making it somewhat similar to languages such as Python. The code is written in a straightforward, imperative style, where lines are executed sequentially. JULIA does not require semicolons at the end of each statement, and blocks of code are defined using keywords like END, in contrast to Python's indentations.

Examples:

```
print("Hello, World!")
```

Or:

```
function greet(name::String)
    return "Hello, " * name
end
```

2.5 Control Abstractions

Julia provides several control abstractions that help manage program execution, including conditional statements, loops, functions, exception handling, and concurrency. Conditional statements use if, elseif, and else, while repetition is handled through for and while loops. Functions are a core part of the language and are created with the function keyword, which makes it easier to organize logic into reusable parts. Julia also supports exception handling with try, catch, and finally, which helps programs recover from errors more safely. In addition, the language includes built-in concurrency features such as asynchronous tasks and threading, which enable tasks to run concurrently.

2.6 Abstraction Features

Julia supports abstraction through functions, structs, mutable structs, modules, packages, and multiple dispatch. These features make it easier to organize a program by separating responsibilities and reducing code duplication. Instead of writing everything in a single long sequence, Julia allows programmers to group logic into smaller, reusable parts that are easier to understand.

Functions are one of the main ways Julia supports abstraction. A function allows a programmer to encapsulate a specific task in its own block of code and then call that code whenever needed. This is useful because it keeps the program organized and avoids repeating the same logic in multiple places. Functions also make a program easier to test and update, since changes can be made in one place instead of throughout the entire program. In Julia, functions are especially important because the language is strongly centered around them, and they work closely with multiple dispatch.

Structs provide another important form of abstraction. A struct lets the programmer create a custom type that groups related data. This makes it easier to model real-world information in a program. For example, instead of storing many separate values with no clear connection, a programmer can define a single structure that represents one object or record. Julia also provides mutable structs, which are useful when the stored values need to be changed after the object is created. This gives programmers flexibility when they need either fixed data or data that can be updated over time.

Modules help support abstraction at a larger level by organizing related code into separate sections. A module can contain functions, types, and other definitions that belong together. This helps keep large programs more manageable and reduces confusion by preventing everything from being placed in a single file or shared space. Modules are especially helpful in larger projects where different parts of the program serve different purposes.

Packages are another abstraction feature in Julia. Packages allow programmers to use libraries that have already been developed for common tasks, such as statistics, data analysis, plotting, or scientific computing. This saves time and allows programmers to build on existing tools rather than writing every feature from scratch.

Packages also make Julia more practical for real-world projects by extending the language beyond its built-in features.

One of Julia's most distinctive abstraction features is multiple dispatch. Multiple dispatch means that the same function name can perform different actions depending on the types of values passed into it. Instead of creating completely separate function names for similar tasks, Julia allows a single function to have several versions, chosen automatically based on the argument types. This supports cleaner program design because related operations can remain under a single shared function name while still handling different data types correctly.

Overall, Julia's abstraction features make it easier to design programs that are clear, reusable, and well-organized. Functions help separate logic into smaller tasks; structs group related data into meaningful types; modules organize larger sections of code; packages provide reusable libraries; and multiple dispatch allows functions to adapt to different kinds of input. Together, these features make Julia a strong language for building programs that are easier to manage and extend.

2.7 Evaluation of the Language

Julia performed well in terms of writability during our project because it allowed us to express our program logic in a direct and organized way. The language made it possible to write conditionals, loops, functions, and custom data structures with minimal extra syntax. This was helpful in our fraud detection project because the system relied on repeated checks, including identifying risky transaction types, comparing balances, calculating fraud scores, and classifying transactions by risk level. Writing these rules in Julia felt fairly natural, and the language made it easier to turn our ideas into working code without excessive complexity. This was especially noticeable in the fraud-scoring portion of the program, where several fraud signals had to be combined into one scoring process.

Julia was also strong in readability, especially once the program was divided into smaller functions and structured data types. In our project, we used functions such as parse_paysim_line, load_csv, fraud_score, and classify_risk, which helped separate the program into clear, distinct tasks. We also used custom structures such as Transaction, CustomerStats, and DestStats, which made the code easier to follow because the names clearly indicated what information each part of the program handled. From our experience, the code became much easier to read when we organized it this way rather than placing everything in one large block. At the same time, there was still a learning curve because Julia was less familiar to us than the languages we had used more often, so some syntax and design choices took time to get used to.

In terms of reliability, Julia helped us build a program that ran consistently and handled errors more safely. One example from our project was the use of exception handling when reading CSV data. If a row was malformed, the program could skip that row instead of crashing the entire run. Typed structures also improved reliability by making the intended data format more explicit and reducing confusion about what kinds of values each part of the program should store. However, our experience also showed that reliability is not only about whether the code runs without crashing. It also depends on the quality of the program's output. In our project, the implementation itself was reliable in the sense that it could process transactions and return results through the backend and dashboard, but the fraud model still had practical limitations, such as producing many medium-risk classifications and being affected by dataset imbalance.

Overall, our experience with Julia was positive. It gave us a good balance between writability and readability while still supporting a reliable implementation for our project. The language allowed us to build a working fraud detection system that connected a Julia script, a Node.js backend, and a browser-based dashboard. Although it took

some time to adjust to the language, Julia was still effective for a project that involved data processing, structured program design, and repeated decision-making logic.

2.8 Strengths and Weaknesses

One major strength of Julia is that it combines high-level readability with strong performance for data-focused work. In our project, this was useful because we needed to process transaction records, apply repeated fraud checks, track account history, and classify suspicious behavior. Julia made it possible to write this logic in a way that stayed fairly clear without feeling overly low-level or difficult to manage. Features such as functions, structs, dictionaries, and loops worked well together and helped us organize the project into separate parts, such as parsing, scoring, classification, and output. From a practical standpoint, Julia felt capable of handling the structured, repetitive processing our project required.

Another strength of Julia is its support for organized program design. The language made it easier for us to model transaction data and account statistics using custom structures, which kept the code more understandable. It also worked well as part of a larger system, rather than as a standalone script. In our project, the Julia code connected to a Node.js backend, which then sent results to a browser dashboard. That made the language feel flexible and practical in a real project setting rather than limited to small examples or isolated calculations.

A practical weakness of Julia is that it may feel less familiar than more commonly used languages such as Java, Python, or JavaScript. During our project, there was a learning curve as we had to adjust to Julia's syntax, its function-centered design, and its general way of organizing code. The language was not overly difficult, but it was not as immediately comfortable as languages we had used more often. This meant that some time had to be spent learning the language itself in addition to building the project.

Another limitation is that while Julia helped us build the system, it did not remove the practical challenges of the problem we were solving. Our fraud detection model still depended on the rules we created, which meant the quality of the output was limited by our scoring choices. For example, the project still produced many medium-risk results, and some evaluation metrics could appear stronger than they actually were due to dataset imbalance. In that sense, Julia was strong as an implementation language, but it did not automatically solve the limitations of a rule-based fraud model.

3. Required Programs Demonstrating Language Features

3.1 Program 1: Data Types and Built-in Methods

Program 1 focused on demonstrating the use of at least four data types and two built-in methods for each type. In our project, these examples were placed in the first section of the Julia file to directly demonstrate the language requirements before the main fraud detection logic begins. The four data types we used were integer, floating-point, string, and Boolean. These examples helped show that Julia supports a range of common data types and built-in operations while also connecting to the larger project.

The first data type we used was the integer type. In the code, we created an integer value and applied the built-in methods abs and iseven. The abs method returned the absolute value of the integer, while iseven checked whether the resulting value was even. These examples demonstrated simple numeric processing and showed how Julia can perform built-in integer operations directly. Although this section was included mainly to satisfy the course requirement, integer values also appear naturally throughout the project in fields such as transaction step numbers, customer counts, and risk counts.

The second data type we used was floating-point. In the code, we created a floating-point value and applied the built-in round and ceil methods. The round method was used to round the number to a specified number of decimal places, while ceil returned the smallest integer greater than or equal to the value. These methods are important in a project like ours because transaction amounts and evaluation metrics often involve decimal values. Floating-point values appear throughout the project in transaction amounts, balances, thresholds, and calculated measures such as precision, recall, and accuracy.

The third data type we used was the string type. In the demonstration section, we created a string value and applied the built-in methods uppercase and replace. The uppercase method converted the text to uppercase, while replace replaced part of the string with a new value. These examples showed how Julia can manipulate text directly. Strings were also an important part of the main project because transaction type names, customer identifiers, destination identifiers, and fraud-reason labels were all stored and processed as text.

The fourth data type we used was Boolean. In the code, we created a Boolean value and applied the built-in NOT (!) operator and xor. The ! operation reversed the truth value, and xor compared two Boolean values to determine whether exactly one was true. This type was especially relevant to our fraud detection project because the dataset includes Boolean-like fraud labels, and the program relies on true-or-false conditions to check whether transactions match suspicious patterns. Boolean values were important both in the demonstration section and in the project's larger fraud-scoring logic.

In addition to the four required data types, we included data manipulation using the sum and mean functions on a list of numbers. This demonstrated Julia's ability to work with collections of numerical data and was relevant to our project because fraud detection often relies on summarizing and comparing transaction behavior over time. Overall, Program 1 demonstrated that Julia supports a wide range of useful built-in methods and that these data types were not only shown in isolation, but also connected to the actual needs of our fraud detection system

3.2 Program 2: Data Structures and Control Structures

Program 2 focused on the main data structures and control structures used in the project. In our fraud detection system, these features were important because the program had to store transaction records, track account history, apply fraud-scoring rules, and classify results in an organized way. Julia supported this well by combining structured storage with clear decision-making logic.

One of the main data structures used in the project was the Transaction struct. This grouped the information for a single transaction into one unit, including the step number, transaction type, amount, sender, balances, destination, and fraud labels. We also used the CustomerStats and DestStats mutable structs to track account behavior over time. CustomerStats stored values such as transaction count, total amount, and risky transaction count for a sender, while DestStats stored destination activity such as count, total amount, and unique senders. These structures were important because the fraud model depended on both individual transaction details and account history.

The project also used vectors and dictionaries. Vectors stored collections such as loaded transactions, suspicious transactions, and classifications returned for output. Dictionaries mapped customer names and destination names to their related statistics, making it possible to retrieve and update account history efficiently as each transaction was processed. The main control structures in the project were conditionals, loops, functions, and exception handling. Conditional statements were used throughout the fraud-scoring logic to check for risky transaction types, large amounts, balance drains, suspicious destination patterns, and other fraud signals. These checks were then used to assign fraud scores and classify transactions as HIGH, MEDIUM, or LOW risk.

Loops were used to process the dataset one transaction at a time, update account statistics, count suspicious behavior, and print selected classifications. Functions helped separate the program into smaller tasks, including parsing CSV rows, loading data, checking suspicious rules, scoring transactions, and classifying risk. Exception handling was used when reading the CSV file so that malformed rows could be skipped instead of stopping the entire program. Overall, the data structures and control structures in Program 2 helped make the system organized and functional. Structs grouped related information, vectors and dictionaries managed collections and account history, conditionals applied the fraud rules, loops processed the dataset, functions separated major tasks, and exception handling improved reliability.

3.3 Program 3: Exception Handling or Concurrency

Our project demonstrates exception handling during CSV loading in the Julia program. When the dataset is read, each line is parsed into a transaction record. This parsing step is placed inside a try-catch block so that if a row is malformed or does not match the expected format, the program can skip that row instead of crashing. In the code, this happens during the file-loading step, where the program attempts to parse each line, adds valid transactions to the list, and prints a message for invalid ones.

This logic matters because the fraud detection system relies on processing many transaction records sequentially. If one bad row caused the program to stop completely, the system would be much less reliable and less practical for handling real data. By using exception handling, the project becomes more fault-tolerant and can continue analyzing the rest of the dataset even when errors appear in individual rows. Exception handling was a better fit for our project than concurrency because the main goal was to ensure the transaction analysis could run safely and consistently. Since the program reads external CSV data and relies on correct parsing, handling input errors was important to the system's overall functionality. This made exception handling a meaningful feature in the project rather than just an isolated example added for the assignment.

4. Problem Definition and Use Cases

4.1 Problem Definition

As digital payments have become more common, financial systems now have to process huge numbers of transactions very quickly. Because of this, it is not realistic for people to manually review every transaction for suspicious activity. Fraud can occur in ways that are easy to miss at first, especially when a transaction appears normal on the surface but follows a pattern unusual for the account or customer. This creates a need for systems that can automatically review transaction data and identify potentially fraudulent activity.

Our project focuses on building a fraud detection system that analyzes transaction data and flags suspicious behavior quickly and efficiently. The system reads transaction records, checks for patterns that may indicate fraud, and returns useful results for the user to review. These patterns can include unusually large transfers, many transfers in a quick succession, or other unusual balance changes.

4.2 System Goals

1. Read and parse a CSV file (with the required headers) to create Transaction objects
2. Put the transaction objects into a list
3. Iterate through the list and detect suspicious transactions
4. Save the suspicious transactions
5. Post to the user

4.3 Use Cases.

1. **Run Fraud Detection Analysis**
   User Action: The user initiates the fraud detection process via the interface or a backend request.
   System Response: The system scans the dataset, applies fraud detection rules, and identifies transactions that appear suspicious.
2. **View Summary Statistics**
   User Action: The user requests an overview of the transaction dataset.
   System Response: The system returns summary information such as total number of transactions, average transaction amount, and counts of suspicious records.
3. **Review Suspicious Transactions**
   User Action: The user wants to see which transactions were flagged during analysis.
   System Response: The system displays the suspicious transactions or provides a count and related details for further review.
4. **Handle Invalid CSV Data**
   User Action: The user submits a CSV file that contains missing values, malformed rows, or incorrect formatting.
   System Response: The system uses exception handling to skip or report invalid lines and, when possible, continues processing valid records.
5. **Display Results in Web Interface**
   User Action: The user clicks a button on the HTML front end to run the analysis.
   System Response: The backend executes the Julia script, parses the output, and displays the results in a readable web-based format.

5. System Design and Proposed Method

5.1 System Architecture

The project uses a simple three-part structure: a frontend, a backend, and a Julia script. The frontend is a web dashboard built with HTML, CSS, and JavaScript. The backend is a small Node.js server that connects the dashboard to the Julia program. The Julia script handles the actual fraud detection logic.

```
app.post("/analyze", (req, res) => {
  execFile("julia", [juliaPath, csvPath, maxRows, threshold, maxPrint], ...);
});
```

Code Snippet

The user clicks the Run Demo button in the browser, which sends a request to the Node.js backend. The backend runs the Julia script on the PaySim-style CSV file, parses the output, and sends the results back to the frontend as JSON. The frontend then displays the returned classifications and updates the charts.

5.2 Frontend Components

The html page is a dashboard that shows the system output. It includes summary cards, a results table, filters, and a small chart. The summary cards show values such as total transactions, suspicious transactions, fraud labels, and model metrics like precision and recall.

```
function renderChart() {

    const canvas = document.getElementById("breakdownChart");

    const fallback = document.getElementById("chartFallback");
```

```
    const highTransfer = allClassifications.filter(item => item.risk === "HIGH" &&
item.type === "TRANSFER").length;

    const highCashOut = allClassifications.filter(item => item.risk === "HIGH" &&
item.type === "CASH_OUT").length;

    const mediumTransfer = allClassifications.filter(item => item.risk === "MEDIUM" &&
item.type === "TRANSFER").length;

    const mediumCashOut = allClassifications.filter(item => item.risk === "MEDIUM" &&
item.type === "CASH_OUT").length;
```

Code Snippet from renderChart() in index.html

The main table displays the flagged transactions along with their risk level, score, and reasons. Filters allow the user to sort by transaction type, risk level, fraud label, and step number.

5.3 Backend Components

The backend is written in Julia, along with a script written in Node.JS that runs and serves the results. Its job is to receive the frontend request, run the Julia script, and convert the returned output into JSON.

```
return {

   transactionsProcessed,uniqueCustomers,

   fraudCount,suspiciousCount,suspiciousTransferCount,

   thresholdUsed,classificationRowsReturned,

   classificationRowsAvailable,highRiskCount,mediumRiskCount,
```

Code Snippet from server.js

The main backend call is /analyze. When called, it runs the Julia file, reads the standard output, and extracts values such as transaction count, fraud count, suspicious count, and classification rows.

5.4 Algorithms or Processing Logic

Each transaction is scored in order. The Julia logic uses factors such as risky transaction type, amount size, balance drain, exact balance drain, destination balance anomaly, and prior sender behavior. The program keeps track of sender and destination history in dicts so each transaction can be scored including context from these prior transactions.

After the score is calculated, the transaction is ranked as low, medium, or high risk. Only TRANSFER and CASH_OUT transactions can become high risk. These are transaction types that are most associated with fraudulent transactions. High-risk transactions are treated as the model's strongest fraud predictions, while medium-risk transactions are transactions that may deserve review.

```
# convert score into risk label
```

```
function classify_risk(t::Transaction, score::Int)

   risky_type = (t.ttype == "TRANSFER" || t.ttype == "CASH_OUT")



   if risky_type && score >= 9

       return "HIGH"

   elseif risky_type && score >= 5

       return "MEDIUM"

   else

       return "LOW"

   end
```

Code Snippet from fraud_detection_demo.jl

6. Implementation

6.1 Development Process

Development was a tedious process across the semester, as learning how to properly score for fraud detection was an added challenge. The model is still not perfect by any means, and could use a lot more fine-tuning and added weights. At first, transactions were only scored on their amount, which is not a reliable sole marker. Week after week, more weights were added and adjusted until we reached this final product.

6.2 Code Organization

The index.html, .JS and .JL files all live in the root directory, while our local dataset files are in a nested /dataset folder.

6.3 Documentation and Commenting

There are comments throughout the code to explain how each function/block of code works. This was helpful during correspondence and collaborative work on the backend, as changing the scoring system became complex and convoluted at times.

```
# Structure customer stats

mutable struct CustomerStats

   count::Int

   total_amount::Float64

   risky_count::Int

end

# Structure destination stats
```

```
mutable struct DestStats
    count::Int
    total_amount::Float64
    unique_senders::Set{String}
```

7. Experiments and Testing

7.1 Experiment Goals

The main goal of our testing was to evaluate whether the fraud detection system worked correctly, and accurately. We wanted to confirm that the system could read the dataset, process transactions in Julia, return results through the Node.js backend, and display those results correctly in the frontend dashboard.

We also wanted to pit our results against results provided by PaySim. This is what our false positives/negatives are revealed by.

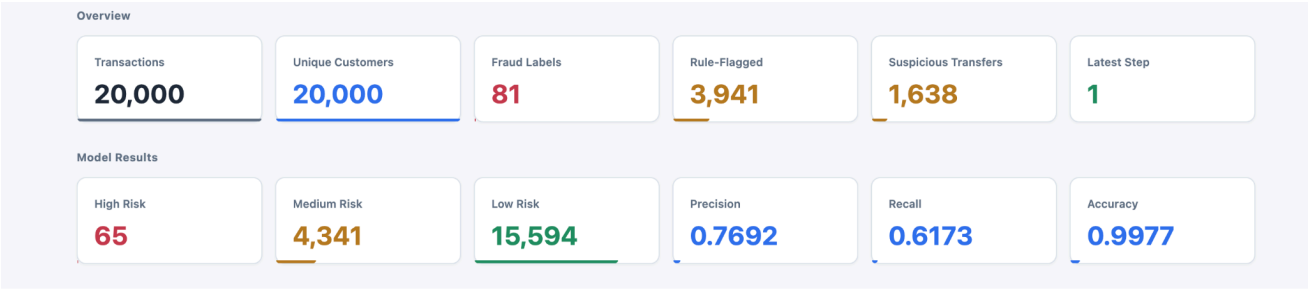
7.2 Test Bed

The project was tested in a local development environment. The frontend was run in Google Chrome, while the backend was run using Node.js and Express. Testing was done on a personal computer in a normal desktop environment rather than on a remote server or high-performance system. The same PaySim dataset was used for all testing.

7.3 Metrics Measured

We measured both model-quality and system-output metrics. On the model side, the most important values were true positives, false positives, true negatives, false negatives, precision, recall, and accuracy. These metrics let us compare the fraud detector's predictions against the isFraud labels already included in the PaySim dataset.

In our final iteration, the system processed **20,000 transactions** and found **81 fraud-labeled transactions** in the dataset. The final model classified **65 transactions as high risk**, **4,341 as medium risk**, and **15,594 as low risk**. It flagged **3,941 transactions** under the original baseline high-transaction-amount rule, of which **1,638** were transfers.



Its final precision was 0.7692, recall was 0.6173, and accuracy was 0.9977.

7.4 Experiment Details and Observations

We tested the system several times as we added more weights to the fraud logic. The earliest versions relied on a simple threshold rule for large TRANSFER and CASH_OUT transactions. The final version uses a more detailed scoring method that considers transaction type, transaction amount, sender balance drain, exact balance drain, destination balance anomalies, and prior history.

The system still tends to overestimate the influence of the total transaction amount, especially for transactions with a short history. Testing also revealed a significant number of false positives, which can skew the result in its favor, even when it misses true fraudulent transactions. In addition, the dataset is extremely large, with a low percentage of this set being classified as fraudulent transactions. This skews the accuracy numbers somewhat, as the ratio means the run can miss many fraudulent transactions while still being seen as moderately accurate. We struggled to implement stronger test cases with a short project timeline.

8. Limitations and Conclusions

8.1 Limitations

The current version of the system serves as a functional fraud-detection demo, but it still has several limitations. One important limitation is that the model produces a large number of MEDIUM-risk classifications. This means the system is often able to identify transactions that may deserve review, but it is not always selective enough to treat them as strong fraud predictions. In practice, this can make the results harder to interpret because many rows are suspicious without being clearly fraudulent.

Another limitation is that the project uses a rule-based scoring model rather than a trained machine learning model. As a result, the system depends entirely on the fraud signals and score weights we selected. While this approach made the project easier to explain and implement, it also means the model does not adapt automatically or improve from additional data. Its performance is limited by the quality of the rules we designed.

The evaluation results also need to be interpreted carefully because the dataset is imbalanced. Our presentation notes that accuracy appears very high, but that metric can be misleading when most records are not fraudulent. In the final results, the model processed 20,000 transactions and reported high accuracy, but it also showed that false positives and false negatives still affect reliability. This means that strong overall accuracy does not automatically mean the fraud detection logic is equally strong in every case.

There are also practical improvements that could make the system more useful. The frontend could be expanded to support uploaded files, stronger charts, and saved results. The model itself could be improved by more carefully tuning the score weights and comparing the current rule-based approach against a machine-learning baseline on the same dataset. These changes would make the project more complete and provide a clearer picture of how effective the current design really is.

8.2 Conclusion

Our team built a complete fraud detection demonstration that combines a Julia-based analysis engine, a Node.js backend, and a browser-based dashboard. The system reads PaySim-style transaction data, applies a rule-based fraud-scoring model, classifies suspicious behavior, and returns results that can be viewed in the frontend. In addition to meeting the course programming requirements, the project gave us experience connecting a language-focused assignment to a practical application.

Through this project, we learned that Julia is a flexible language that works well for structured data handling, repeated scoring logic, and organized program design. Features such as functions, structs, mutable structs, dictionaries, and exception handling made it possible to build the core fraud detection workflow in a way that stayed readable and manageable. We also learned that Julia can be used as part of a larger system rather than only as a standalone script, since our project successfully connected the Julia program to a backend and web interface.

At the same time, the project showed us that a language alone does not solve every problem. Julia helped us implement the system effectively, but the quality of the final output still depended on our model design, scoring choices, and interpretation of the dataset. Even with those limits, the language was a strong fit for the project because it allowed us to build something practical while learning a different programming style from the ones we normally use.

Overall, we would use Julia again for projects that involve structured data, repeated analysis, or numerical processing. Our experience showed that it is capable, organized, and useful for data-focused applications. For this project, Julia was a worthwhile choice because it helped us create a working system while also giving us meaningful experience with a language outside our usual stack.

8.3 References

Cite documentation, research papers, articles, or other sources used during the project.

Chesnes, M., & Nguyen, S. T. (2026, March 25). *FTC testifies before the Joint Economic Committee on Agency's efforts to combat fraud*. Federal Trade Commission. https://www.ftc.gov/news-events/news/press-releases/2026/03/ftc-testifies-joint-economic-committee-agency s-efforts-combat-fraud?

Lopez-Rojas, E., & Axelsson, S. (2016).
PaySim: A financial mobile money simulator for fraud detection.
Proceedings of the 28th European Modeling and Simulation Symposium.

Bezanson, J., Edelman, A., Karpinski, S., & Shah, V. B. (2017).
Julia: A fresh approach to numerical computing. SIAM Review, 59(1), 65–98.
https://doi.org/10.1137/141000671

Ngai, E. W. T., Hu, Y., Wong, Y. H., Chen, Y., & Sun, X. (2011).
The application of data mining techniques in financial fraud detection: A classification framework and an academic review of literature.
Decision Support Systems, 50(3), 559–569.
https://doi.org/10.1016/j.dss.2010.08.006

Holy, T. E., Sargent, T. J., & Steele , G. L. (2016). *Juliacon 2016*. JuliaCon 2016. https://juliacon.org/2016/